Authenticated but Unauthorized: 5D H-NGAC for Bounded-Latency Authorization in FPGA-Based Real-Time Robotics

Hassan Karim*, Md Omar Faruque†, Abdel-Hameed A. Badawy‡, Sai Sitharaman§, Deepti Gupta¶
*Stable Cyber LLC †New Mexico State University ‡Florida International University
§Zetafence, Inc. ¶Texas A&M University - Central Texas

***Abstract*—Real-time robotic and drone system designers often leave authorization out of the control path because software policy engines do not offer decisions within a fixed cycle budget. Operating system scheduling, not policy evaluation, creates non-deterministic latency, so a faster software-based engine does not offer the guarantees required by hard real-time applications. The gap leaves attack classes open that identity-based authorization cannot see, including unsafe-state operation and command injection from compromised but credentialed nodes. State of the art H-NGAC compiles NGAC policy graphs into fixed-width hardware bitmasks so that an authorization decision reduces to a chain of bitwise AND operations. In this paper we extend H-NGAC with two further dimensions, runtime system state and command provenance. We also synthesize these four- and five-dimensional kernels to a Zynq-7020 FPGA. We evaluate the synthesized kernels against six software baselines, and the five-dimensional policy in an adversarial Robot Operating System 2 (ROS 2) scenario against a compromised node that has already authenticated. We found that the added security dimension is free in time. Both kernels resolve in an identical number of clock cycles at every policy size tested, with a closed-form worst case and zero jitter by construction. In software, the same added dimension makes every policy rule 19% more expensive. Thus, the authorization worst case becomes a synthesis parameter rather than a measurement, and we quantify what that determinism costs in area and in average-case speed.**

***Index Terms*—access control, NGAC, authorization, FPGA, hardware security, cyber-physical systems, embedded systems, real-time systems, bounded latency, command provenance, ROS 2**

I. Introduction

A quadruped inspection robot in maintenance mode should not move. Consider Layla, a field technician replacing a battery pack on a 45 kg platform when a velocity command arrives from a remote operator console. The DDS transport authenticates the console's certificate and delivers the command in well under a millisecond. No layer of that stack asks whether a remote operator is entitled to command motion while a technician holds the interlock. If the platform acts, Layla absorbs the consequence. This failure mode is not hypothetical. CVE-2022-33323 documents unauthorized command execution against Mitsubishi MELFA industrial robots [1], and CVE-2021-38425 documents message injection against eProsima Fast DDS, the default ROS 2 transport [2].

Robotics platforms leave authorization out of the control path for a defensible reason. Although modern policy engines evaluate rich attribute-based policy, they cannot bound the decision. That is, the latency tail of a software policy decision point is governed by operating system scheduling, preemption and cache state, not by policy evaluation, so a faster engine does not tighten the bound. In our own published measurements, an OPA edge deployment took 1 to 5 ms per decision and an XACML decision point near 50 ms, while a software authorization path averaging 1.14 μs still produced a single 157 μs scheduling outlier [3]. A further evaluation, now under review, measured OPA at a 271.491 μs mean with 13 missed real-time deadlines [4]. A control loop cannot budget for a tail it cannot bound. Thus engineers place authorization at the perimeter and leave the control path open.

Our research line approaches this as a compilation problem. The hypergraph privilege analysis of Sitharaman et al. established that large privilege structures compile into forms that answer set-level questions quickly [5], and that insight is the intellectual origin of this work. H-NGAC, presented at IEEE DCAS 2026, carried the idea to the decision path: compile the NGAC policy graph [6], [7] into fixed-width bitmasks so that one authorization decision reduces to a chain of bitwise AND operations [3]. H-NGAC checked the identity triple of subject, object and attribute. It did not model the platform's runtime state, and it did not model where a command came from. Layla's interlock lives in exactly those two missing dimensions.

In this paper we extend H-NGAC with those two dimensions and synthesize the result to reconfigurable logic. We contribute three things: (a) 5D H-NGAC, an authorization model that adds a system-state mask σ and a command-provenance mask π to the H-NGAC identity triple, closing the two CVE-anchored attack classes above at the application layer where transport security cannot see them; (b) a synthesized hardware evaluation on a Zynq-7020 showing that the four- and five-dimensional kernels resolve in an identical number of clock cycles at every policy size tested, with a closed-form latency of $12 + n/2$ cycles for n rules and zero jitter by construction, while the fifth dimension costs 524 LUTs and no BRAM or DSP; and (c) an adversarial ROS 2 evaluation in which the five-dimensional policy blocks 18,878 injection attempts from a node holding valid credentials with zero false positives across 17,059 legitimate commands, together with a six-model software comparison showing that every baseline that beats H-

NGAC on mean latency does so by over-authorizing.

The central finding deserves its own sentence. In software, each added security dimension makes every policy rule more expensive, raising the per-rule cost from 0.645 to 1.220 cycles between the three- and five-dimensional variants. In hardware the added dimensions cost zero cycles. They are free in time and nearly free in area, which converts the authorization worst case from a measurement into a synthesis parameter.

The remainder of this paper is organized as follows. Section II describes the threat model and the background this work extends. Section III formalizes the 5D H-NGAC decision function. Section IV renders the model into a pipelined hardware architecture and derives its determinism. Section V details the experimental method and its honesty rules. Section VI reports results. Section VII rates related work on a scored rubric. Section VIII bounds our claims, and Section IX concludes.

II. Background and Threat Model

A. From DAG traversal to bitmask evaluation

NGAC, standardized as INCITS 565, expresses policy as a directed graph of users, objects and attributes, and answers an access query by reachability traversal [6], [7]. The generality is valuable and the traversal is the cost. H-NGAC compiles the policy graph ahead of time into fixed-width bitmasks so the runtime decision is a chain of AND operations over precomputed sets [3]. We refer to the traversal-based reference model as DAG-NGAC throughout. This paper extends H-NGAC and inherits its compilation step unchanged.

B. Attack classes

We consider three named attack classes, each anchored to a published CVE against deployed industrial or robotic systems.

Unauthorized access. The attacker acts as an unprivileged agent or through a hijacked session (CVE-2022-45789, Schneider Electric Modicon [8]). The identity triple closes this class, and it is the class every access control model addresses.

Unsafe-state operation. The attacker, or a confused legitimate client, issues an otherwise well-formed command while the platform is in a state that makes the command hazardous: battery low, maintenance mode, calibration required (CVE-2022-33323, Mitsubishi MELFA [1]). Identity-based authorization admits this class because the request is legitimate on every identity axis.

Command provenance abuse. The attacker holds valid DDS credentials for an authorized subject but is not an entitled source type for the command it issues (CVE-2021-38425, eProsima Fast DDS [2]). SROS2 and DDS-Security authenticate who a node is at the transport layer [9], [10]. They never ask whether that source type may issue that command to that resource. Further, the transport layer's own access control is fragile: Deng et al. demonstrated four ways a credentialed insider bypasses SROS2 permissions and keeps publishing [11]. That is an application-layer question, and it is invisible at the transport layer by design.

A fourth concern is deliberately not an attack class. The timing window, in which a correct deny arrives after the actuator has already acted, is a delivery property that cuts across all three classes. It is closed by bounding decision latency below message propagation time, not by any policy dimension. We return to it in Section VI.

C. Assumptions

We assume the following. The attacker holds valid transport credentials for at least one authorized subject and can publish at wire rate. Transport security itself is intact. We do not defend broken cryptography. The policy decision point and the compiled policy are trusted. Policy compilation happens off the decision path, and policy updates are outside our present scope. The platform exposes its runtime state to the decision point as a bit vector.

III. The 5D H-NGAC Model

We formalize the decision function before describing hardware. Let U be the universe of policy nodes with $|U| = 256$. A set over U is a bitmask $M \in \{0,1\}^{256}$, stored as four 64-bit words.

Definition 1 (Policy rule). A policy rule is a five-tuple

$$r_i = (S_i,\, O_i,\, A_i,\, \sigma_i,\, \pi_i) \tag{1}$$

where $S_i, O_i, A_i \in \{0,1\}^{256}$ are the permitted subject, object and attribute sets, $\sigma_i \in \{0,1\}^{32}$ enumerates the state bits that must be asserted for the rule to apply, and $\pi_i \in \{0,1\}^{32}$ enumerates the source types entitled to invoke it. A zero σ_i or π_i is a wildcard.

Definition 2 (Request). A request is a five-tuple $q = (s,\, o,\, a,\, \sigma_q,\, \pi_q)$ with subject identifier $s \in \mathbb{Z},\ 0 \le s < 256$, object identifier $o \in \mathbb{Z},\ 0 \le o < 256$, required attributes $a \in \{0,1\}^{256}$, the platform's current state vector $\sigma_q \in \{0,1\}^{32}$, and the requester's provenance $\pi_q \in \{0,1\}^{32}$.

Definition 3 (Match and decision). Rule r_i matches request q when

$$\begin{aligned} match_i(q) = S_i[s] \wedge O_i[o] \wedge (a \subseteq A_i) \\ \wedge\ (\sigma_i \subseteq \sigma_q) \wedge (\pi_i = 0 \vee \pi_i \cap \pi_q \neq \emptyset) \end{aligned} \tag{2}$$

and the decision over a policy of n rules is

$$\delta(q) = \bigvee_{i=0}^{n-1} match_i(q), \qquad \delta \in \{0,1\}. \tag{3}$$

Every term in (2) is a bitwise AND against a precompiled mask followed by a comparison. That is, the first three terms are the H-NGAC identity triple from our prior work, the σ term conditions the permission on the platform's runtime state, and the π term conditions it on the command's source type. Dimensions four and five add two more AND terms to a conjunction that already exists. They add no traversal, no lookup and no branch. One scope statement belongs here. We compile the association subset of INCITS 565, that is, the rules that grant. Prohibitions, obligations and administrative relations are not compiled, and Section VIII bounds our claims accordingly.

Hypothesis. An added security dimension is free in time if and only if its evaluation synthesizes to additional combinational inputs within an existing pipeline stage, rather than to an additional sequential stage or memory access. Equation (2) is constructed so that σ and π satisfy this condition: each is one AND-and-compare against a 32-bit register, far narrower than the 256-bit identity terms beside it. Section VI tests this hypothesis directly.

The limit behavior of (3) matters for the reader budgeting a control loop. As n grows toward the 512-rule capacity, hardware cost grows only through scan length, and Section IV shows that growth is exactly half a cycle per rule regardless of how many dimensions each rule carries. In software the same growth compounds with dimensionality: our measured per-rule slope rises from 0.645 cycles in three dimensions to 1.220 cycles in five. Thus the two curves diverge precisely where policy gets rich.

IV. Hardware Architecture

The kernel evaluates (3) as a pipelined linear scan. Each clock, a two-stage pipeline loads the next two rules from policy memory while evaluating the two rules loaded on the previous clock, giving 0.5 cycles per rule at an initiation interval of one. The policy resides behind a BRAM-style port and the request and rule count arrive over AXI-Lite registers. Synthesis maps the evaluation entirely to LUTs and flip-flops. The kernel consumes zero BRAM and zero DSP blocks, so the policy port is the only memory interface.

Determinism is structural, not observed. The scan executes $\lceil n/2 \rceil + 1$ pipeline iterations regardless of where, or whether, a match occurs. A permit latches the decision but does not shorten the scan, and an odd rule count pads with an all-zero rule that cannot match. Thus minimum, mean and maximum latency are equal by construction, and the measured closed form

$$L(n) = 12 + n/2 \text{ cycles} \tag{4}$$

holds exactly at every policy size we tested, where the constant 12 covers interface handshake and pipeline fill. At 100 MHz, $L(500) = 262$ cycles is 2.62 μs, and the 512-rule capacity bounds the primitive at 2.68 μs. A designer needing a tighter bound shrinks n or raises the clock, and either way the bound is known before deployment. That is what we mean by the worst case becoming a synthesis parameter.

We note one design consequence plainly. Forgoing early exit costs the average case: a software scan that exits on first match would beat (4) whenever the match lands early. We spend that average case to buy a worst case with zero variance, because the control loop budgets for the worst case and only the worst case.

V. Experimental Method

A. Platforms

Hardware results come from Vitis HLS 2025.2 targeting the Zynq-7020 (xc7z020-clg400-1, the PYNQ-Z1 part) at a 10 ns clock. Cycle counts are taken from Verilog co-simulation transaction reports, which are cycle accurate. Functional correctness is additionally verified on PYNQ-Z1 silicon. Software results come from an Intel i7-12800H running the same kernel code compiled with g++ -O3, measured with Linux perf over 200,000 iterations after 1,000 warmup iterations per model. The ROS 2 evaluation runs on ROS 2 Jazzy.

TABLE I
Kernel cycles per decision (Verilog co-simulation). Min, mean and max are equal at every point. 4D and 5D are identical columns.

Rules	Requests	4D cycles	5D cycles
4	11	14	14
10	27	17	17
50	134	37	37
100	267	62	62
200	534	112	112
500	1,334	262	262

B. Request corpus

The corpus generator emits, for each policy rule, one request that satisfies all five dimensions, plus a state-failing and a provenance-failing variant where applicable, yielding eleven requests for the four-rule policy and 1,334 at 500 rules. The corpus therefore exercises permit paths, state denials and provenance denials at every policy size, and it is the corpus under which the correctness columns of Section VI are computed.

C. Honesty rules

We label every number by how it was obtained, and we hold the paper to the labels. Hardware cycle counts are measured and deterministic. No confidence interval applies because variance is zero by construction. Software per-decision cycle counts are derived: perf supplies a measured effective clock of 4.96 GHz, and cycles are mean nanoseconds multiplied by that clock. They are not per-decision counter reads. Proportions from the adversarial run carry exact denominators and one-sided 95% Wilson score bounds. One baseline, RBAC with a modeled external state lookup, is a synthetic busy-wait. We exclude it from every empirical claim and use the measured SQLite-backed variant instead. The timing-window measurement produced zero slips and is reported only as a null result. Board verification is functional only, and we cite no timing from silicon.

VI. Results

A. The key finding: dimensionality is free in time

Table I reports co-simulated kernel cycles per decision. The four- and five-dimensional kernels resolve in an identical number of cycles at every policy size, with identical initiation interval and identical 0.33 ns timing slack, and with minimum, mean and maximum equal at every point. The closed form (4) fits all six points exactly. Adding the provenance dimension to the state-aware kernel costs zero clock cycles.

TABLE II
RESOURCE UTILIZATION FROM SYNTHESIS (ZYNQ-7020, 100 MHZ).

Metric	4D	5D	Delta
II	1	1	0
Iteration latency	3	3	0
LUT	4,580 (8%)	5,104 (9%)	+524 (+11.4%)
FF	2,579 (2%)	2,679 (2%)	+100 (+3.9%)
BRAM	0	0	0
DSP	0	0	0
Timing slack	0.33 ns	0.33 ns	0

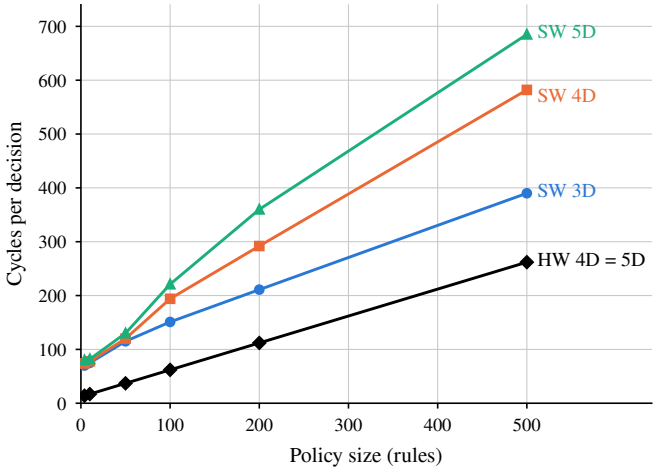

Fig. 1. Cycles per decision versus policy size. Software cycles are derived from perf-measured means and the 4.96 GHz clock; hardware cycles are co-simulated. Software slopes steepen with each added dimension. The 4D and 5D hardware series are identical at every measured point.

The dimension is not free in area, and we do not claim it is. Table II shows the fifth dimension costs 524 LUTs (+11.4%) and 100 flip-flops (+3.9%), with zero BRAM and zero DSP in both kernels. The five-dimensional kernel occupies 9% of a low-cost part. Free in time, nearly free in area. An earlier four-dimensional build also completed Vivado place and route with +2.170 ns worst negative slack and zero failing endpoints of 1,987, so the co-simulated clock is not optimistic for this fabric.

B. *In software, the same dimension is not free*

The identical kernel logic, compiled for the i7 and measured under perf, tells the opposite story. Per-rule marginal cost rises from 0.645 cycles in three dimensions to 1.024 in four and 1.220 in five. That is, in software every rule pays for every dimension on every decision, and the cost compounds with policy size: at 500 rules the five-dimensional software path spends 685 derived cycles per decision against the hardware kernel's 262 measured cycles. Figure 1 plots both families. The software curves fan out as dimensions are added. The hardware lines for four and five dimensions are the same line.

C. *Faster baselines are faster because they are wrong*

Table III compares six software models on the same corpus. The RBAC hash map resolves in roughly 100 derived cycles

TABLE III
SOFTWARE MODELS, DERIVED CYCLES PER DECISION AT 500 RULES, AND CORPUS CORRECTNESS. "OVER" = ADMITS STATE- OR PROVENANCE-VIOLATING REQUESTS.

Model	Cycles (500)	Decides correctly?
H-NGAC 3D	390	No, over-authorizes
H-NGAC 4D	582	No, over-authorizes
H-NGAC 5D	685	**Yes**
RBAC hash map	100	No, over-authorizes
DAG-NGAC traversal	1,032	No, over-authorizes
Flattened 5D lookup	984	**Yes**, 19.7× memory

at every policy size, and the flat cost is genuine. However, it admits 200,000 of 200,000 requests in the April corpus, including all 100,000 that violate state, and the DAG-NGAC and three-dimensional paths admit the same. A model that cannot see the state or provenance dimensions cannot deny on them. The only software models that decide correctly are the five-dimensional path and a flattened direct-lookup table that trades a 19.7× memory footprint for its speed. However, an RBAC deployment could encode state into its role set, a maintenance operator role distinct from a normal operator role. Sixteen state bits and three source types multiply that role count combinatorially, and the honest alternative, RBAC consulting an external state store, cost 20.6 times the four-dimensional path when we measured it against an in-process SQLite store. It might be tempting to read Table III as an argument that cheap static authorization suffices. The correctness column is the answer: the fast rows are fast because they answer a smaller question than the one Layla's interlock asks.

D. *Adversarial evaluation in ROS 2*

We ran the provenance-abuse scenario live. A software gatekeeper node executing the same five-dimensional decision function fronted an actuator topic while a legitimate node and a compromised node both published with valid credentials for the same authorized subject, the compromised node at five times the legitimate rate. Over a 30-second session the policy blocked 18,878 of 18,878 injection attempts, a block rate of 100% with a one-sided 95% Wilson lower bound of 99.99%, and passed 17,059 of 17,059 legitimate commands, a false positive rate of 0% with a 95% upper bound of 0.02%. This demo and the board run carry two separate claims. The demo establishes what the five-dimensional policy semantics block under sustained adversarial load. The silicon run establishes that the synthesized kernel computes identical decisions on all 2,307 corpus requests. Neither claim substitutes for the other. Under three- or four-dimensional policy every one of those injections would have been admitted, because the attacker's identity triple and state context were valid. Only the provenance dimension separates the two traffic streams.

The same runs measured the timing-window property. Under both idle and saturated CPU load, zero authorization callbacks of 8,733 exceeded the 50 μs lower bound of DDS localhost propagation. We report this strictly as a null result.

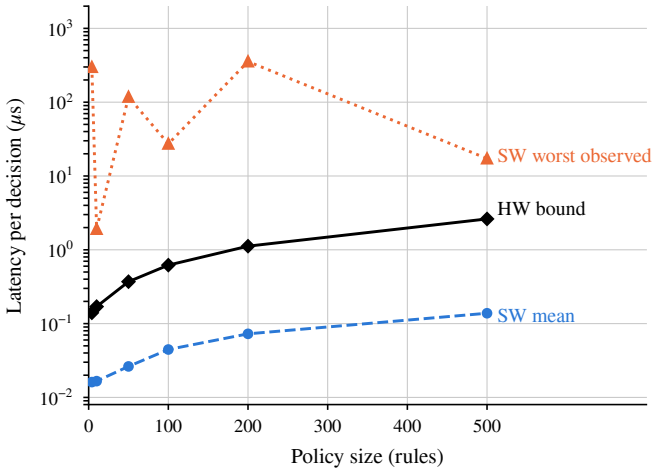

Fig. 2. Wall-clock latency per decision, log scale. The software mean and worst observed come from perf measurements of the five-dimensional path. The hardware line is the closed-form bound of (4) at 100 MHz, where minimum, mean and maximum coincide. The software worst case is an observation, not a bound, and does not track policy size.

The architectural argument stands on (4): a synthesis-time bound below the propagation floor closes the window by construction, which no measured software mean can do.

E. The comparison we do not make, and the one we do

It might be tempting to infer from Table I that the fabric outpaces the CPU. That inference would be incorrect, and we pre-empt it. At 500 rules the i7 completes a mean five-dimensional decision in 138 ns of wall clock against the fabric's 2.62 μs, because a 4.96 GHz core is 49.6 times faster per cycle than a 100 MHz fabric. The CPU wins the mean by roughly 19×. The defensible comparison is the worst case, and there the ordering reverses: the software path's worst observed decision at 200 rules was 355.4 μs against the fabric's invariant 1.12 μs, and its worst at four rules was 298.8 μs against 0.14 μs. Further, the software figure is an observation, not a bound. The 157 μs outlier in our DCAS baseline arrived in a run whose mean was 1.14 μs [3]. An operating system does not guarantee its own tail, and a control loop budgets for the tail. Figure 2 plots both concessions on one wall-clock axis. The software mean runs below the hardware bound at every policy size. The software worst case runs above it at every size, by more than three orders of magnitude at four rules, and it does not track policy size, because the tail belongs to the scheduler and not to the policy.

For completeness we place general-purpose policy engines on the same axis. In our prior evaluation, Open Policy Agent averaged 271.491 μs per decision with 13 deadline misses [4], and an XACML OpenPDP deployment averaged roughly 50 ms [3], [12]. These engines evaluate far richer policy than (2) expresses, and we do not fault their generality. We observe only that they occupy a latency regime three to five orders of magnitude above the closed form of (4), which is the regime that keeps authorization out of control paths.

TABLE IV
RELATED WORK SCORED ON THE FOUR RUBRIC AXES.

Prior work	*pdl*	*wcb*	*dim*	*hwv*
DAG-NGAC / Policy Machine [6], [7]	1	0	2	0
Linear-time NGAC [13]	3	1	2	0
XEngine [14]	3	0	3	0
OPA [15]	3	0	3	0
XACML PDP [12]	2	0	3	0
SROS2 / DDS-Security [9], [10]	2	0	0	0
App-layer ROS authorization [16]	2	0	1	0
ROS-Defender [17]	2	0	1	0
ABAC for multi-robot systems [18]	2	0	3	0
FPGA reference monitor [19]	3	3	1	4
NoC data protection units [20]	3	3	1	3
Bit-vector classification [21], [22]	4	4	0	4
SoC access-control wrappers [23]	3	3	1	4
CHERI [24]	4	4	0	5
Security-aware RT scheduling [25], [26], [27]	3	2	1	0
CPS runtime enforcement [28]	2	3	1	0
HyperNGAC [5]	0	0	3	0
H-NGAC [3]	4	2	0	1
TS-NGAC [4]	4	3	3	0
This work	**5**	**5**	**5**	**5**

VII. RELATED WORK

We rate prior work on four axes, each an integer score $x \in \mathbb{Z},\ 0 \leq x \leq 5$: per-decision latency evidence (pdl), where 5 requires cycle-accurate per-decision measurement; worst-case boundedness (wcb), where 5 requires a closed-form bound verified at every measured point; enforcement dimensionality beyond the identity triple (dim), where 3 denotes one added runtime dimension and 5 denotes both state and provenance; and hardware validation (hwv), where 5 requires cycle-accurate co-simulation plus silicon functional verification. Table IV scores each line of prior work under these definitions. The paragraphs below justify the scores that matter most, and they concede three genuine ancestors plainly.

Software policy engines and models. The NGAC standard and the Policy Machine define the model we compile but report no per-decision latency evidence [6], [7], [29], and NGAC's decision and review advantages over XACML are established in the standards comparison of Ferraiolo et al. [30]. NGAC implementations to date are software, including NIST's own fine-grain enforcement over database queries [31], and the strongest software NGAC decision result is the linear-time graph algorithms of Mell et al. [13]. That is the right software bound, and it is still a per-decision traversal whose constant depends on the host. XEngine compiles XACML into numeric structures offline and speeds decisions by orders of magnitude [14], the closest software analogue of our compilation step, but its decision latency still varies with policy and request shape. Servos and Osborn's survey lists ABAC decision performance among the field's open problems [32], and our own measurements place OPA and an XACML PDP three to five orders of magnitude above (4) [15], [4], [12].

Hardware enforcement. This literature holds three genuine ancestors, and we cite them as such. Huffmire et al. compiled a formally specified memory-isolation policy directly into an

FPGA reference monitor in 2006 [19], and Fiorin et al. placed access-rights checks into NoC interfaces with no added latency on the authorized path [20]. Both make enforcement latency a property of a synthesized circuit. Neither evaluates a relational policy model, and neither analyzes what an added policy dimension costs. The third concession is packet classification: TCAM and bit-vector engines match wide multi-field keys at deterministic rates [21], and the StrideBV line advertises latency independent of ruleset content [22]. However, StrideBV latency grows linearly with match-key width. That is, in the closest deterministic prior art, added dimensions are precisely not free in time, which is the axis this paper claims. Aker wraps SoC controllers in verified access-control hardware at the bus-transaction level [23], and CHERI enforces per-instruction memory capabilities in silicon [24]. Both police a lower layer than a policy model.

Real-time and robot security. The real-time security literature schedules software enforcement around deadlines: Contego slots security tasks into scheduling slack [25], Lesi et al. trade control quality against integrity checking [26], and the 2024 SoK catalogs the paradigm [27]. We exit that paradigm rather than optimize within it, since a fabric decision consumes no CPU slack at all. Pinisetty et al. synthesize runtime enforcers from timed automata for reactive CPS [28], the nearest formal-synthesis neighbor, targeting safety properties rather than authorization and offering no hardware latency model. In robotics, Dieber et al. argued for application-layer authorization above transport crypto with identity-based decisions [16], ROS-Defender filters node flows at the network layer [17], and Salimi et al. condition per-request ABAC decisions on task and capability attributes through a blockchain bridge [18]. None conditions a command on its source type or the platform's runtime safety state, and none is an inline bounded-latency primitive.

Since this work measures cycle-accurate per-decision latency, verifies a closed-form bound at every measured point, enforces both state and provenance, and validates through co-simulation and silicon functional verification, we assign it the maximum on all four axes under the definitions above. We state the novelty claim in its bounded form. Constant-latency wide-key matching exists in TCAMs, deterministic classification exists in the bit-vector family, and policy-to-circuit compilation dates to 2006. To our knowledge, no prior work compiles a standardized relational authorization model to hardware and demonstrates, on identical toolflow and part, that added policy dimensions cost zero clock cycles, and no prior hardware implementation of NGAC exists at all. We invite correction on both points.

VIII. Limitations

Our claims are bounded by several factors, and we state them plainly. First, the three-dimensional kernel was never synthesized and its kernel code no longer exists, thus every hardware parity claim in this paper is scoped to four versus five dimensions. Second, no on-board timing exists. Cycle counts come from cycle-accurate co-simulation, and the silicon run verifies function only. Third, the software baseline is a 4.96 GHz laptop CPU while the fabric runs at 100 MHz; an embedded-class CPU baseline on the board's own Cortex-A9 would make the wall-clock comparison fairer to the fabric, and we did not run it. Fourth, software latency distributions were measured under WSL2, whose scheduler jitter contaminates maxima, which is why we cite worst observed values and never call them WCETs. Fifth, the primitive is capacity-bounded at 512 rules and 256 policy nodes, and policy update happens off the decision path. Sixth, one baseline, the modeled external-state lookup, is synthetic, and we excluded it from every empirical claim after finding it added nothing the measured SQLite path does not show. Seventh, the kernel compiles only the association subset of INCITS 565. Prohibitions, obligations and administrative relations are not implemented, so no claim here extends beyond rules that grant. Eighth, the zero-cycle result is demonstrated at one operating point, 100 MHz on a Zynq-7020, where both kernels close timing with identical +0.33 ns slack. At an aggressive clock target the added dimension could force an additional pipeline stage, which is exactly the boundary the hypothesis of Section III predicts. Characterizing that frequency ceiling is future work.

IX. Future Work and Conclusion

Four questions structure our next steps. 1. Can a re-derived three-dimensional kernel restore the full three-way parity claim in synthesis? 2. What measurement path yields trustworthy on-board cycle counts on Zynq-class parts? 3. Does the closed form of (4) survive a dynamic policy update path, or does update traffic reintroduce jitter? 4. How does the primitive compose with time-scoped delegation, which our journal-track work develops in software? Further, we call on the community to report authorization performance as cycle-level worst cases rather than means, because a mean is precisely the statistic a control loop cannot budget against.

In conclusion, we extended the H-NGAC bitmask primitive with system-state and command-provenance dimensions and synthesized both the four- and five-dimensional kernels to a Zynq-7020. The added dimension costs zero clock cycles and 524 LUTs, latency is closed-form at $12+n/2$ cycles with zero jitter by construction, and the five-dimensional policy blocked 18,878 of 18,878 credentialed injection attempts in ROS 2 with zero false positives in 17,059 legitimate commands. Robotics platform engineers and functional-safety engineers can treat the authorization worst case as a synthesis parameter, place the decision inside the control loop budget, and close the two attack classes that identity-based authorization admits. Layla's interlock becomes a policy bit the fabric checks in bounded time, not a procedure she hopes the operator follows.

References

[1] National Vulnerability Database, "CVE-2022-33323: Mitsubishi electric MELFA unauthorized command execution," https://nvd.nist.gov/vuln/detail/CVE-2022-33323, 2022.

[2] ——, "CVE-2021-38425: eProsima fast DDS RTPS message injection," https://nvd.nist.gov/vuln/detail/CVE-2021-38425, 2021.

[3] H. Karim, S. Sitharaman, and D. Gupta, "Hardware-accelerated NGAC authorization for real-time multi-robot systems," in *2026 IEEE 19th Dallas Circuits and Systems Conference (DCAS)*, 2026, pp. 1–4.

[4] H. Karim, D. Gupta, and S. Sitharaman, "Deterministic time-scoped NGAC for real-time multi-robot systems," 2026, under review.

[5] S. Sitharaman, H. Karim, D. Gupta, and M. Tyagi, "Scalable privilege analysis for multi-cloud big data platforms: A hypergraph approach," in *2025 IEEE International Conference on Big Data (BigData)*, 2025, pp. 6626–6633.

[6] InterNational Committee for Information Technology Standards, "Information technology — next generation access control (NGAC)," ANSI/INCITS, Tech. Rep. INCITS 565-2020, 2020.

[7] D. Ferraiolo, V. Atluri, and S. Gavrila, "The Policy Machine: A novel architecture and framework for access control policy specification and enforcement," *Journal of Systems Architecture*, vol. 57, no. 4, pp. 412–424, 2011.

[8] National Vulnerability Database, "CVE-2022-45789: Schneider electric modicon session hijack," https://nvd.nist.gov/vuln/detail/CVE-2022-45789, 2022.

[9] V. Mayoral-Vilches, R. White, G. Caiazza, and M. Arguedas, "SROS2: Usable cyber security tools for ROS 2," in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022.

[10] Object Management Group, "DDS security specification, version 1.1," OMG, Tech. Rep., 2018.

[11] G. Deng, G. Xu, Y. Zhou, T. Zhang, and Y. Liu, "On the (In)Security of secure ROS2," in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2022, pp. 739–753.

[12] OASIS, "eXtensible access control markup language (XACML) version 3.0," OASIS Standard, Tech. Rep., 2013.

[13] P. Mell, J. Shook, R. Harang, and S. Gavrila, "Linear time algorithms to restrict insider access using multi-policy access control systems," *Journal of Wireless Mobile Networks, Ubiquitous Computing, and Dependable Applications (JoWUA)*, vol. 8, no. 1, 2017.

[14] A. X. Liu, F. Chen, J. Hwang, and T. Xie, "Designing fast and scalable XACML policy evaluation engines," *IEEE Transactions on Computers*, vol. 60, no. 12, pp. 1802–1817, 2011.

[15] Open Policy Agent Project, "Open policy agent," https://www.openpolicyagent.org, 2024, cNCF graduated project.

[16] B. Dieber, B. Breiling, S. Taurer, S. Kacianka, S. Rass, and P. Schartner, "Security for the robot operating system," *Robotics and Autonomous Systems*, vol. 98, pp. 192–203, 2017.

[17] S. Rivera, S. Lagraa, C. Nita-Rotaru, S. Becker, and R. State, "ROS-Defender: SDN-based security policy enforcement for robotic applications," in *Proc. IEEE Security and Privacy Workshops (SPW)*, 2019, pp. 114–119.

[18] S. Salimi, F. Keramat, J. P. Queralta, and T. Westerlund, "A customizable conflict resolution and attribute-based access control framework for multi-robot systems," *Journal of Systems Architecture*, vol. 168, p. 103528, 2025.

[19] T. Huffmire, S. Prasad, T. Sherwood, and R. Kastner, "Policy-driven memory protection for reconfigurable hardware," in *Computer Security – ESORICS 2006, LNCS 4189*, 2006, pp. 461–478.

[20] L. Fiorin, G. Palermo, S. Lukovic, V. Catalano, and C. Silvano, "Secure memory accesses on networks-on-chip," *IEEE Transactions on Computers*, vol. 57, no. 9, pp. 1216–1229, 2008.

[21] H. Song and J. W. Lockwood, "Efficient packet classification for network intrusion detection using FPGA," in *Proc. ACM/SIGDA 13th Int. Symp. on Field-Programmable Gate Arrays (FPGA '05)*, 2005, pp. 238–245.

[22] T. Ganegedara, W. Jiang, and V. K. Prasanna, "A scalable and modular architecture for high-performance packet classification," *IEEE Transactions on Parallel and Distributed Systems*, vol. 25, pp. 1135–1144, 2014.

[23] F. Restuccia, A. Meza, and R. Kastner, "Aker: A design and verification framework for safe and secure SoC access control," in *Proc. IEEE/ACM Int. Conf. on Computer-Aided Design (ICCAD)*, 2021, pp. 1–9.

[24] R. N. M. Watson, J. Woodruff, P. G. Neumann, S. W. Moore *et al.*, "CHERI: A hybrid capability-system architecture for scalable software compartmentalization," in *2015 IEEE Symposium on Security and Privacy*, 2015.

[25] M. Hasan, S. Mohan, R. Pellizzoni, and R. B. Bobba, "Contego: An adaptive framework for integrating security tasks in real-time systems," in *Proc. 29th Euromicro Conf. on Real-Time Systems (ECRTS), LIPIcs 76*, 2017, pp. 23:1–23:22.

[26] V. Lesi, I. Jovanov, and M. Pajic, "Security-aware scheduling of embedded control tasks," *ACM Transactions on Embedded Computing Systems*, vol. 16, no. 5s, pp. 188:1–188:21, 2017.

[27] M. Hasan, A. Kashinath, C.-Y. Chen, and S. Mohan, "SoK: Security in real-time systems," *ACM Computing Surveys*, vol. 56, no. 9, pp. 218:1–218:31, 2024.

[28] S. Pinisetty, P. S. Roop, S. Smyth, N. Allen, S. Tripakis, and R. von Hanxleden, "Runtime enforcement of cyber-physical systems," *ACM Transactions on Embedded Computing Systems*, vol. 16, no. 5s, pp. 178:1–178:25, 2017.

[29] V. C. Hu, D. Ferraiolo, R. Kuhn, A. Schnitzer, K. Sandlin, R. Miller, and K. Scarfone, "Guide to attribute based access control (ABAC) definition and considerations," NIST, Tech. Rep. Special Publication 800-162, 2014.

[30] D. Ferraiolo, R. Chandramouli, R. Kuhn, and V. Hu, "Extensible access control markup language (XACML) and next generation access control (NGAC)," in *Proc. 2016 ACM Int. Workshop on Attribute Based Access Control (ABAC '16)*, 2016, pp. 13–24.

[31] D. F. Ferraiolo, S. I. Gavrila, G. Katwala, and J. D. Roberts, "Imposing fine-grain next generation access control over database queries," in *Proc. 2nd ACM Workshop on Attribute-Based Access Control (ABAC '17)*, 2017.

[32] D. Servos and S. L. Osborn, "Current research and open problems in attribute-based access control," *ACM Computing Surveys*, vol. 49, no. 4, pp. 65:1–65:45, 2017.